

强化学习在推荐系统中的应用综述

Executive Summary 强化学习应用到推荐系统的核心价值，是把“一次打分”扩展为“多步决策”：系统不只优化当前点击，而是优化会话质量、转化、停留、复访、留存等长期价值。近十年方法演进大致经历了：上下文 Bandit 处理在线探索；MDP/深度 RL 处理多步反馈；Slate/List-level RL 处理列表依赖；Model-based 与 Offline RL 处理样本效率和上线安全；IRL/偏好学习处理 reward 难设计；Multi-agent、Hierarchical 与 Imitation 则分别解决多模块协同、超大动作空间与弱监督序列生成。工程上并不存在“一招鲜”：工业系统更常见的是混合范式，即用 Bandit 做安全探索、用离线 RL 或 world model 学长期策略、用 Actor-Critic 或 Seq2Slate 做列表生成、再用约束/守护指标控制上线风险。

1 问题建模与范式总览

推荐系统中的 RL 通常把用户状态记为 s_t （画像、上下文、历史序列、会话阶段），动作记为 a_t （单物料、候选集合、整页/整列、频道或多模块决策），奖励记为 r_t （点击、观看时长、加入购物车、转化、会话深度、复访、留存或其组合），目标是最大化折扣回报 $\mathbb{E} \sum_t \gamma^t r_t$ 。与监督式推荐不同，RL 的关键不在于更强的打分器，而在于：其一，允许把推荐视为交互闭环；其二，允许显式建模探索-利用、短期-长期、多目标约束与列表级 credit assignment。

应用方式总览 从工程视角，RL 在推荐中的主要落地方式可概括为九类：基于 Bandit 的在线学习、基于 MDP 的策略学习（policy gradient、Q-learning、actor-critic）、基于 slate/list-level RL、基于模型的 RL（model-based）、基于逆强化学习/奖励学习、基于离线 RL 与反事实评估、基于多智能体/对抗式 RL、基于层次 RL/选项框架、以及基于模仿学习/序列生成（Seq2Slate 等）。这些范式并非互斥，而是沿着“动作空间大小、是否需要在线探索、reward 是否容易定义、是否要优化整页/整链路”四条轴线组合出现。

2 在线与模型无关方法

范式一：基于 Bandit 的在线学习 核心思想是把推荐看作单步决策，在每次曝光时进行探索与利用折中；典型算法包括 ϵ -greedy、UCB、LinUCB、Thompson Sampling 与神经 contextual bandit。它最适合冷启动、召回层探索、样本到达快而反馈近实时的场景。优点是上线链路短、在线可解释性强、A/B 容易；缺点是通常只看即时奖励，难处理跨会话长期价值、列表依赖和 delayed reward；工程难点在于 propensity 记录、探索预算控制、冷启动安全和高方差 OPE。

代表工作中, *A Contextual-Bandit Approach to Personalized News Article Recommendation* (Li et al., 2010) 是工业推荐中 contextual bandit 的里程碑: 状态是用户与文章上下文特征, 动作为待展示新闻, 奖励为点击, 在线采用 LinUCB, 在 Yahoo! Front Page today module 上验证了 contextual exploration 的有效性。 *Unbiased Offline Evaluation of Contextual-Bandit-based News Article Recommendation Algorithms* (Li et al., 2011) 进一步给出 replay/offline replay 评估, 为日志驱动的反事实评估奠定基础。更近期的 *Deep Exploration for Recommendation Systems* (Zhu & Van Roy, 2021/2023) 把推荐从单步 bandit 推向“探测式”多步探索, 强调 delayed feedback 与 sparse reward 下的 probing; *Epinet for Content Cold Start* (Jeon et al., 2024) 则把近似 Thompson Sampling 扩展到复杂神经网络, 并在 Facebook Reels 冷启动 retrieval 场景做了线上验证。实践上, Bandit 最常被放在召回和探索桶, 而不是直接替代精排策略。

范式二: 基于 MDP 的策略学习 (policy gradient、Q-learning、actor-critic) 该类方法把推荐视为多步决策过程, 显式优化长期回报。典型算法包括 REINFORCE、DQN/Double DQN、DDPG/TD3、A2C/A3C、SAC 及其离线变体。适用场景是 feed、短视频、音频流、会话电商等, 用户反馈会改变随后的状态分布。优点是能处理长期目标; 缺点是样本效率较低、对状态建模敏感、容易受离线分布偏移影响; 工程难点在于超大离散动作空间、负反馈稀疏、价值高估和安全探索。

在 policy gradient 路线中, *Top-K Off-Policy Correction for a REINFORCE Recommender System* (Chen et al., 2019) 展示了 YouTube 生产环境中如何把 REINFORCE 用于 top-K 推荐, 状态是用户历史与上下文, 动作是 top-K 候选生成/选择, 训练上使用 off-policy correction 与 top-K correction, 说明 PG 可在超大候选空间中落地。Q-learning 路线中, *Reinforcement Learning to Optimize Long-term User Engagement in Recommender Systems* (Zou et al., 2019, FeedRec) 以 hierarchical LSTM 建 Q-network, 并引入 S-network 作为环境辅助网络, 强调长短期混合反馈与 off-policy 稳定性。Actor-Critic 路线中, *Off-Policy Actor-Critic for Recommender Systems* (Chen et al., 2022) 面向生产系统总结了 critic 设计、分布偏移缓解、离线到在线迁移; *Two-Stage Constrained Actor-Critic for Short Video Recommendation* (Cai et al., 2023) 则把短视频推荐建模为 CMDP: 主目标是 watch-time/长期收益, 约束是 like/share/follow 等辅助行为, 展示了 RL 在多目标约束优化中的工业价值。学术侧还有 *Self-Supervised Reinforcement Learning for Recommender Systems* (Xin et al., 2020) 与 *Supervised Advantage Actor-Critic for Recommender Systems* (Xin et al., 2022), 其贡献在于把 supervised sequential learning 与 RL head 结合, 缓解纯离线 RL 的稀疏奖励与负样本缺失问题。

3 列表生成与模型化方法

范式三: 基于 slate/list-level RL 该范式直接把一个列表、一页、甚至带布局的页面看作动作, 优化 top-K 依赖、多样性、互补性与位置偏置。典型方法包括 list-wise actor-critic、SlateQ、latent-slate RL、page-wise RL。它适合精排/重排、全页生成、feed 卡片编排等场景。优点是目标与业务展示单元一致; 缺点是组合动作空间爆炸、credit assignment 困难; 工程挑

战集中在 slate 表示、用户选择模型假设、离线训练的 combinatorial generalization 以及线上 latency。

Deep Reinforcement Learning for Page-wise Recommendations (Zhao et al., 2018, Deep-Page) 把状态定义为用户实时反馈与历史，动作为带 2-D 展示结构的一整页，强调“选什么”与“怎么摆”的联合优化。*Seq2Slate: Re-ranking and Slate Optimization with RNNs* (Bello et al., 2018) 采用序列生成/pointer network 逐步选择列表元素，用 click log 的弱监督端到端学习 intra-slate dependencies；从方法论上看，它既属于 list-level RL，也属于序列生成式推荐。*Reinforcement Learning for Slate-based Recommender Systems: A Tractable Decomposition and Practical Methodology* (Ie et al., 2019, SlateQ) 在温和选择假设下，把 slate 的长期价值分解成 item-wise LTV，并在 YouTube live experiments 验证了可扩展性，是工业 slate RL 的代表。面向更一般的整列表建模，*Generative Slate Recommendation with Reinforcement Learning* (Deffayet et al., 2023, GeMS) 先用 VAE 学习 slate latent action，再在连续潜空间做 RL，从而减少对 SlateQ 分解假设的依赖；作者公开了代码 (<https://github.com/naver/gems>)。

范式四：基于模型的 RL (Model-based) Model-based RL 通过学习用户环境模型、reward model 或 world model，让智能体先在模拟器中交互，再低风险迁移到线上。适用场景是在线探索代价高、真实奖励延迟长、需要 replay/counterfactual 验证的系统。优点是样本效率高、可做 stress test 与 safe pre-training；缺点是 world model 偏差会被策略放大 (model exploitation)；工程难点在于 simulator fidelity、uncertainty estimation、sim-to-real gap 与 reward calibration。

Generative Adversarial User Model for Reinforcement Learning Based Recommendation System (Chen et al., 2019, GAUM) 用 GAN 学习用户行为动态与奖励，再配合 cascading DQN 做组合推荐，说明 user model 可作为可重复交互的环境近似。*Model-Based Reinforcement Learning with Adversarial Training for Online Recommendation* (Bai et al., 2019, IRecGAN) 进一步把生成器、判别器与策略学习耦合起来，用 adversarial training 校正生成轨迹和奖励偏差。*Whole-Chain Recommendations* (Zhao et al., 2020, DeepChain) 把多场景推荐链路建成多智能体+model-based 框架：不同场景 agent 共享用户记忆，联合最大化整条链路收益，适合首页-详情页-二跳链路这种“全链路优化”。而 *Sim-to-Real Interactive Recommendation via Off-Dynamics Reinforcement Learning* (Wu et al., 2021) 与其扩展版 DACIR (2022) 关注现实中的 simulator mismatch，提出 dynamics adaptation 和 representation adaptation，把 world model 从“能跑”推进到“可迁移”。

范式五：基于逆强化学习/奖励学习 在很多推荐任务里，真正想优化的是满意度、复访、长期留存或体验公平，但这些目标很难手工写成准确 reward，因此可以反过来从偏好、演示或行为轨迹中学习 reward。优点是减轻 reward engineering；缺点是需要额外的人类偏好或优质轨迹，且 reward model 本身可能偏；工程难点在于偏好采样成本、reward drift、噪声演示与离线偏差放大。

Generative Inverse Deep Reinforcement Learning for Online Recommendation (Chen et al., 2020/2021, InvRec) 是推荐领域较早的 rec-specific IRL 工作，核心是从用户行为轨迹

中自动恢复 reward，而不是手工拼接点击/停留等代理指标；实验使用 VirtualTB 在线环境。*PrefRec: Recommender Systems with Human Preferences for Reinforcing Long-term User Engagement* (Xue et al., 2023) 把奖励学习推进到 preference-based recommendation：用用户历史或 crowdsourcing 得到的 pairwise preference 训练 reward model，再用 expectile regression 与额外 value function 学策略，很适合“长期目标难定义、但人能比较两段体验谁更好”的场景。*Adversarial Batch Inverse Reinforcement Learning: Learn to Reward from Imperfect Demonstration for Interactive Recommendation* (Liu et al., 2023) 进一步考虑 demonstration 不完美和 coverage 不充分的问题，引入 pessimism、stationary distribution correction 与 KL 约束。最后，*Reinforcement Learning-based Recommender Systems with Large Language Models for State Reward and Action Modeling* (Wang et al., 2024) 展示了另一条前沿路线：把 LLM 当作环境或语义先验，用于状态摘要、reward shaping 和 action semantic modeling，本质上也是神经奖励/环境建模。

4 离线学习、反事实评估与扩展范式

范式六：基于离线 RL 与反事实评估 离线 RL 直接从日志数据学习策略，是工业推荐最现实的训练范式之一。它最适合探索受限、用户体验敏感、系统已有大量历史曝光日志的场景。优点是安全、成本低、能利用海量旧数据；缺点是 OOD action value 估计困难；工程难点在于 logging policy 不可得、support 不足、奖励稀疏和 OPE 高方差。

A General Offline Reinforcement Learning Framework for Interactive Recommendation (Xiao & Wang, 2021) 是推荐离线 RL 的代表工作，提出 support constraints、supervised regularization、policy constraints、dual constraints 和 reward extrapolation，以缓解行为策略与目标策略的分布偏移。*Text-Based Interactive Recommendation via Offline Reinforcement Learning* (Zhang et al., 2022) 结合 conservative Q-function 与 behavior-agnostic off-policy correction，面向多未知行为策略来源的日志，是对交互推荐/OPE 设计的有价值补充。*Offline Evaluation for Reinforcement Learning-based Recommendation: A Critical Issue and Some Alternatives* (Deffayet et al., 2023) 指出“next-item prediction 式离线评估”并不能真正检验 RL 推荐器的长期优势，提醒工程上不能用普通序列推荐协议替代 RL evaluation。最新的 *ROLeR: Effective Reward Shaping in Offline Reinforcement Learning for Recommender Systems* (Zhang et al., 2024) 把重点放在 reward shaping 与 uncertainty penalty 上，公开了代码 (<https://github.com/ArronDZhang/ROLeR>)，是当前 model-based offline RL for RecSys 的代表之一。配套资源方面，*RL4RS* (Wang et al., 2023) 提供真实数据、模拟环境与 counterfactual policy evaluation 算法，*EasyRL4Rec* (Yu et al., 2024) 则提供统一训练/评估框架和五个数据环境，适合作为工业前的实验基座。

范式七：基于多智能体/对抗式 RL 当推荐由多个模块、多个场景、多个业务 agent 协同完成时，单 agent RL 往往无法表达真实系统；这时可以把场景、模块、频道或多个利益相关方视为协作/竞争智能体。优点是贴合系统结构；缺点是非平稳性更强、训练更难；工程挑战在于 credit assignment、通信约束、集中训练分散执行 (CTDE) 以及多目标协调。

Learning to Collaborate: Multi-Scenario Ranking via Multi-Agent Reinforcement Learning (Feng et al., 2018) 把多场景排序建模成 fully cooperative partially observable MARL, 通过 centralized critic 与消息通信提升整体链路收益。*Learning to Collaborate in Multi-Module Recommendation via Multi-Agent Reinforcement Learning without Communication* (He et al., 2020) 面向电商页面多个独立模块无法直接通信的现实, 设计 signal network 实现弱耦合作。Whole-Chain Recommendations (Zhao et al., 2020) 可视为 MARL 在连续推荐链路中的工业化变体。对话推荐中的 *CRSAL: Conversational Recommender Systems with Adversarial Learning* (Ren et al., 2020) 则把 adversarial learning 融入 Actor-Critic, 以提高对话动作生成质量, 代表了对抗式 RL 在推荐中的一条支线。

范式八：基于层次 RL/选项框架 层次 RL 通过高层策略做粗粒度规划、低层策略做细粒度决策, 缓解超大动作空间、长时延回报和稀疏反馈问题。它适合综合推荐、频道-物料两级决策、category-to-item 生成及多时间尺度目标。优点是更可扩展、可解释; 缺点是层级设计本身较难, 若上层目标错误会系统性误导下层。

Hierarchical Reinforcement Learning for Integrated Recommendation (Xie et al., 2021, HRL-Rec) 是代表性工作: 高层为频道/源选择, 低层为子频道内 item 推荐, 适合 heterogeneous item sources 的 integrated recommendation, 代码公开 (<https://github.com/modriczhang/HRL-Rec>)。 *Hierarchical Reinforcement Learning for Modeling User Novelty-Seeking Intent in Recommendation* (Li et al., 2023) 用层次 RL 建模 novelty-seeking, 使多样性/新颖性进入长期决策回路。 *Hierarchical Reinforcement Learning for Temporal User Adaptation in List-wise Recommendation* (Ji et al., 2024/2025, mchRL) 则从时间抽象角度处理 list-wise recommendation 中长期感知与短期兴趣变化的冲突。更偏选项 (options) 框架的 *Personalised Multi-modal Interactive Recommendation with Hierarchical Reinforcement Learning* (Wu et al., 2024, PMMIR) 把个性化多模态交互拆成顺序子任务, 体现了 options 在交互推荐中的实用性。

范式九：基于模仿学习/序列生成 (Seq2Slate 等) 当日志足够丰富而在线探索昂贵时, 直接模仿“专家用户”或“优质策略”, 常比从零开始 RL 更稳健。优点是样本效率高、训练稳定; 缺点是容易复制历史偏差、难超越 demonstration; 工程挑战在于 covariate shift、专家选择偏差与与 RL 微调衔接。

Energy-Based Sequence GANs for Recommendation and Their Connection to Imitation Learning (Yoo et al., 2017) 较早指出推荐中的生成式序列学习与 MaxEnt imitation learning 的联系。前文介绍的 Seq2Slate (Bello et al., 2018) 也可归入“弱监督序列生成+策略学习”范式: 它不直接优化手工 reward, 而是用点击日志近似地学习“专家式”排列。前沿工作 *Stratified Expert Cloning for Retention-Aware Recommendation at Scale* (Lin et al., 2025, SEC) 直接从高留存用户中构造 expert strata, 用 hierarchical behavior cloning 优化 retention, 并完成大规模 A/B; 它提醒我们, 在长期价值场景中, 高质量 demonstration 本身可能比手工 reward 更可靠。因此工业实践常用“BC/Seq2Slate 预训练 + Offline RL/Preference RL 微调”的两阶段方案。

5 范式比较

表 1: 各范式的工程比较

范式	动作空 间规模	列表级 优化	样本效 率	适合离 线训练	是否需 在线探 索	reward 设 计难度	工程复杂 度	典型应用场景
Bandit 在线学习	中	弱	高	中	必需但 可控	低-中	低	冷启动、召回探索、新闻/短内容单步推荐
MDP 策略学习	大	中	中-低	中	通常需 要或需 强 OPE	中-高	中-高	Feed、短视频、音频流、多步会话推荐
Slate/List-level RL	极大	强	低	中	通常需 要模拟 器/离 线预训	高	高	精排重排、整页生成、布局优化
Model-based RL	大	强	高	强	可极少 上线探 索	高（需环 境/奖励模 型）	高	探索昂贵、需要仿真验证、全链路优化
IRL/奖励学习	大	中-强	中	强	可选	显式设计 低，但偏 好采集高	高	满意度/留存等难定义目标，人工偏好可得
Offline RL + OPE	大	中-强	中	最强	尽量少	中-高	高	真实工业日志学习、安全预训练、上线前评估
多智能体/对抗式 RL	大	中-强	中-低	中	常需安 全探索	高	很高	多模块页面、多场景链路、对话推荐协同
层次 RL/选项框架	很大	强	中	中-强	可少量	中-高	高	频道-物料两级决策、语义到物料 coarse-to-fine
模仿学习/序列生成	大	强	高	强	可不需 要	低-中	中	日志充足、探索昂贵、序列重排/列表生成

6 方法演进时间线

timeline

title 强化学习在推荐系统中的重要里程碑

2010 : LinUCB / Contextual Bandit for Yahoo News

2011 : Replay-based Unbiased Offline Evaluation

2018 : DeepPage / TPGR / REINFORCE@YouTube

2018 : Seq2Slate (sequence generation for slate)

2019 : FeedRec / GAUM / IRecGAN / SlateQ

2020 : Whole-Chain / MARL for modules & scenarios / InvRec

2021 : HRL-Rec / General Offline RL for Interactive Recommendation

2022 : Off-Policy Actor-Critic / SA2C / Text-based Offline RL / CMDP for short video

2023 : PrefRec / Generative Slate RL / Critical Offline Evaluation

2024 : ROLeR / EasyRL4Rec / LLM as environment for state-reward-action modeling
2025 : SEC / HSRL / 更强的 retention-oriented imitation & semantic hierarchical RL

7 工程实践建议

Reward 设计 推荐系统中的 reward 应从业务目标树出发，而不是从单一点击率出发。可行做法是把目标分成三层：第一层是原子信号（click、watch-time、like、share、purchase、return）；第二层是业务代理目标（会话深度、GMV、复访、留存、创作者生态、满意度）；第三层是守护约束（退订率、跳失率、投诉率、时延、资源成本、公平性）。常见实现包括线性加权、分段折扣、延迟归因、约束 MDP、双 critic 或多 critic。若 reward 难以写准，应优先考虑 preference/reward model；若只关心安全探索，则 reward 尽量保持简单、稳定、可解释。一个经验法则是：把不可逆伤害放入约束，把可权衡收益放入主目标。

离线评估与反事实校准 离线评估不能只看 next-item HR/NDCG。对于 RL 推荐，应同时做四类检查：第一，时间切分与 logging policy 完整性检查，确保没有数据/特征穿越；第二，*counterfactual evaluation*，优先使用 replay (bandit)、IPS/SNIPS/DR、长期 OPE 和分群后的 OPE；第三，环境回放或 world model stress test，检验策略是否利用了 simulator bug；第四，策略行为分析，包括推荐分布是否塌陷、cover rate、多样性、新老物料曝光占比、对各人群是否存在异常漂移。离线指标最好同时报告即时与长期，如 CTR、CVR、watch-time、session return、retention proxy 与 guardrail。

安全上线 (guardrail) 推荐 RL 最忌讳“直接全量替换”。建议采用“**behavior-regularized pretraining** → **小流量 canary** → **分群放量**”路径。策略侧可加 KL/BC 正则、action mask、candidate filter、uncertainty penalty、safe fallback；实验侧要提前设定 hard guardrails，如投诉率、跳失率、停留时长底线、内容风险比例、系统时延和成本上界。对可能影响生态分配的场景，应同步监控 item-side fairness、创作者分发集中度和新内容冷启动损失。若存在明显非平稳性（节假日、热点事件、算法切换），应保持旧策略 shadow run，并支持实时回切。

分群/分流策略 RL 很少在全量用户上均匀生效。常见分群维度包括新老用户、活跃度、冷/热物料、内容垂类、流量入口、终端网络条件、价格敏感度和用户生命周期阶段。工程上常见做法是：冷启动与探索桶用 bandit，新用户用 imitation/behavior-regularized policy，老用户高价值群体用长期价值 RL，整页重排用 Slate/Seq2Slate，跨模块或跨场景用 MARL/HRL。分流时则采用“高置信低风险群体先放量，易受损高价值群体后放量”的原则。

范式选择与混合策略 若目标主要是单步点击提升，优先 Bandit；若目标是会话级长期收益，优先 MDP/Actor-Critic；若目标是整页或整列表质量，优先 Slate/List-level 或 Seq2Slate；若探索昂贵、日志充足，优先 Offline RL + OPE；若 reward 难定义，优先 Preference/IRL；若系统是多模块、多场景，优先 MARL；若动作空间极大或需要 *coarse-to-fine*，优先 Hierarchical RL。工业上推荐的混合范式通常是：**Bandit 做在线探索 + Offline RL 做长期价值学习 +**

Actor-Critic/Seq2Slate 做列表生成 + CMDP/guardrail 做安全约束。这类混合策略比纯单范式更符合真实推荐架构。

结论 强化学习应用到推荐系统，并不是“把排序模型换成 RL”，而是用决策视角重写推荐系统的优化目标与训练闭环。从 Bandit 到 MDP，从 SlateQ/Seq2Slate 到 world model 与 offline RL，再到 preference learning、MARL 和 HRL，方法演进本质上都在回答同一组工程问题：如何在超大动作空间中安全探索，如何用离线日志学习长期价值，如何把难以书写的业务目标转化为可靠 reward，如何让整页、整链路而不是单条样本变得最优。若只求短期点击，RL 未必必要；但当业务开始关心留存、体验、公平、生态与多目标约束时，RL 往往是最自然的统一框架。面试或技术分享中，最有说服力的回答不是背算法，而是能够说明：你为何选择某一范式，它解决了哪一类推荐问题，又如何与离线评估、在线安全和业务目标对齐。

8 SlateQ 算法

SlateQ 是一种面向推荐列表场景的强化学习算法，其核心目标是解决推荐系统中 **组合动作空间过大** 的问题。在传统强化学习中，智能体在状态 s 下选择一个动作 a ，并通过动作价值函数 $Q(s, a)$ 评估该动作的长期收益。然而，在推荐系统中，系统通常不是向用户推荐单个物品，而是一次展示一个由多个物品组成的推荐集合，即 slate。若候选物品集合为 $\mathcal{I}$ ，每次推荐 k 个物品，则一个推荐动作可以表示为

$$A = \{i_1, i_2, \dots, i_k\}, \quad A \subseteq \mathcal{I}, \quad |A| = k.$$

此时动作空间规模为

$$\binom{|\mathcal{I}|}{k},$$

当候选物品数量很大时，直接对每一个推荐集合学习 $Q(s, A)$ 几乎不可行。SlateQ 的基本思想是：不直接学习整个推荐列表的价值，而是将推荐列表的长期价值分解为用户可能选择的单个物品的长期价值，再通过用户选择模型组合得到整个 slate 的价值。

8.1 推荐列表强化学习建模

在 SlateQ 中，推荐过程可以建模为一个马尔可夫决策过程。状态 s_t 表示用户在时刻 t 的兴趣状态、历史行为、上下文信息等；动作 A_t 表示系统展示给用户的推荐列表；用户看到推荐列表后，会从列表中选择某个物品 $i \in A_t$ ，也可能不选择任何物品，记为 $\perp$ 。交互之后，系统得到即时奖励 r_t ，并转移到下一状态 s_{t+1} 。推荐系统的优化目标是最大化长期折扣回报：

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^T \gamma^t r_t \right],$$

其中 $\gamma \in [0, 1]$ 为折扣因子， π 为推荐策略。

如果直接将整个推荐列表 A 作为动作，则动作价值函数为

$$Q^\pi(s, A) = R(s, A) + \gamma \sum_{s'} P(s' | s, A) V^\pi(s'),$$

其中 $R(s, A)$ 表示在状态 s 下展示列表 A 的即时奖励期望, $P(s' | s, A)$ 表示展示列表 A 后转移到下一状态 s' 的概率, $V^\pi(s')$ 表示后续状态价值。该形式虽然符合标准强化学习定义, 但由于 A 是组合动作, 直接学习和优化 $Q(s, A)$ 的代价极高。

8.2 SlateQ 的核心假设

SlateQ 能够进行价值分解, 依赖于一个关键假设, 即 **奖励和状态转移主要依赖于用户最终选择的物品**。也就是说, 用户看到推荐列表 A 后, 虽然系统展示了多个物品, 但真正影响即时奖励和后续状态的, 主要是用户最终点击、观看或消费的那个物品。

设用户在状态 s 下看到列表 A 后选择物品 i 的概率为

$$P(i | s, A),$$

其中 $i \in A$ 。若用户没有选择任何物品, 则记为 $\perp$ 。在该假设下, 列表级即时奖励可以写成

$$R(s, A) = \sum_{i \in A} P(i | s, A) R(s, i),$$

其中 $R(s, i)$ 表示用户选择物品 i 后产生的即时奖励期望。类似地, 状态转移概率可以写成

$$P(s' | s, A) = \sum_{i \in A} P(i | s, A) P(s' | s, i).$$

这意味着整个推荐列表的影响可以被表示为不同用户选择结果的加权平均。

8.3 SlateQ 的价值分解

根据上述假设, 列表动作价值函数可以展开为

$$Q^\pi(s, A) = R(s, A) + \gamma \sum_{s'} P(s' | s, A) V^\pi(s').$$

将奖励分解和状态转移分解代入上式, 有

$$Q^\pi(s, A) = \sum_{i \in A} P(i | s, A) R(s, i) + \gamma \sum_{s'} \sum_{i \in A} P(i | s, A) P(s' | s, i) V^\pi(s').$$

交换求和顺序后可得

$$Q^\pi(s, A) = \sum_{i \in A} P(i | s, A) \left[R(s, i) + \gamma \sum_{s'} P(s' | s, i) V^\pi(s') \right].$$

定义物品级长期价值函数为

$$Q^\pi(s, i) = R(s, i) + \gamma \sum_{s'} P(s' | s, i) V^\pi(s'),$$

则列表级价值函数可以分解为

$$Q^\pi(s, A) = \sum_{i \in A} P(i | s, A) Q^\pi(s, i).$$

该公式是 SlateQ 的核心。它表明，推荐列表的长期价值不是列表中各个物品价值的简单相加，而是由用户选择概率加权后的期望长期价值。

从直观上看， $Q^\pi(s, i)$ 表示用户真正选择并消费物品 i 后带来的长期价值，而 $P(i | s, A)$ 表示物品 i 在当前列表中被用户选择的概率。因此，一个物品即使长期价值很高，如果用户几乎不会点击它，它对当前推荐列表的贡献也可能很小；相反，一个物品即使点击概率很高，但长期价值较低，也可能会抢占其他高价值物品的点击概率，从而降低整体长期收益。

8.4 用户选择模型

为了计算

$$P(i | s, A),$$

SlateQ 需要引入用户选择模型。常用的一类选择模型是多项 Logit 模型，即 MNL 模型。设物品 i 在状态 s 下的吸引力为

$$v(s, i) > 0,$$

用户不选择任何物品的吸引力为

$$v(s, \perp) > 0.$$

那么用户选择物品 i 的概率可以写为

$$P(i | s, A) = \frac{v(s, i)}{v(s, \perp) + \sum_{j \in A} v(s, j)}.$$

其中分母

$$v(s, \perp) + \sum_{j \in A} v(s, j)$$

表示用户所有可选项的总吸引力，包括不点击任何物品以及点击推荐列表中的某个物品。因此，该分母本质上是一个归一化项，用来保证所有选择概率之和为 1：

$$P(\perp | s, A) + \sum_{i \in A} P(i | s, A) = 1.$$

其中

$$P(\perp | s, A) = \frac{v(s, \perp)}{v(s, \perp) + \sum_{j \in A} v(s, j)}.$$

将 MNL 选择模型代入 SlateQ 的价值分解式，可以得到

$$Q(s, A) = \sum_{i \in A} \frac{v(s, i)}{v(s, \perp) + \sum_{j \in A} v(s, j)} Q(s, i).$$

由于分母对于列表中的所有物品是相同的，因此可以写成

$$Q(s, A) = \frac{\sum_{i \in A} v(s, i) Q(s, i)}{v(s, \perp) + \sum_{j \in A} v(s, j)}.$$

该式说明，SlateQ 优化的并不是单纯的点击率，也不是单纯的物品长期价值，而是点击概率和点击后长期价值共同决定的期望收益。

8.5 Slate 优化问题

在得到物品级价值函数 $Q(s, i)$ 和用户选择模型 $P(i | s, A)$ 后, 推荐系统需要在候选物品集合 $\mathcal{I}$ 中选择一个大小为 k 的推荐列表 A , 使得列表级价值最大:

$$A^* = \arg \max_{A \subseteq \mathcal{I}, |A|=k} Q(s, A).$$

在 MNL 选择模型下, 该问题可以写成

$$A^* = \arg \max_{A \subseteq \mathcal{I}, |A|=k} \frac{\sum_{i \in A} v(s, i) Q(s, i)}{v(s, \perp) + \sum_{j \in A} v(s, j)}.$$

为了将该问题写成规划形式, 可以为每个候选物品 $i \in \mathcal{I}$ 引入二元变量

$$x_i = \begin{cases} 1, & \text{物品 } i \text{ 被选入推荐列表,} \\ 0, & \text{物品 } i \text{ 未被选入推荐列表.} \end{cases}$$

则推荐列表大小约束为

$$\sum_{i \in \mathcal{I}} x_i = k.$$

目标函数可以写成

$$\max_x \frac{\sum_{i \in \mathcal{I}} x_i v(s, i) Q(s, i)}{v(s, \perp) + \sum_{j \in \mathcal{I}} x_j v(s, j)},$$

约束条件为

$$\sum_{i \in \mathcal{I}} x_i = k, \quad x_i \in \{0, 1\}, \quad \forall i \in \mathcal{I}.$$

这是一个分式整数规划问题。其困难在于, 选择某个物品不仅会增加分子中的长期价值贡献, 同时也会增加分母中的总吸引力, 从而改变列表中其他物品的选择概率。因此, SlateQ 中的列表优化并不能简单退化为选择 $Q(s, i)$ 最大的 k 个物品。

在一些条件下, 可以将二元变量松弛为连续变量:

$$0 \leq x_i \leq 1,$$

从而得到分式线性规划:

$$\begin{aligned} & \max_x \frac{\sum_{i \in \mathcal{I}} x_i v(s, i) Q(s, i)}{v(s, \perp) + \sum_{j \in \mathcal{I}} x_j v(s, j)}, \\ & \text{s.t.} \quad \sum_{i \in \mathcal{I}} x_i = k, \quad 0 \leq x_i \leq 1. \end{aligned}$$

该松弛形式可以降低求解难度, 并为大规模推荐系统中的近似 slate 优化提供基础。

8.6 SlateQ 的训练过程

SlateQ 的训练数据通常来自用户与推荐系统的历史交互。每条样本可以表示为

$$(s_t, A_t, c_t, r_t, s_{t+1}),$$

其中 s_t 为当前用户状态, A_t 为系统展示的推荐列表, c_t 为用户最终选择的物品, r_t 为即时奖励, s_{t+1} 为下一状态。如果用户没有点击或消费任何物品, 则有 $c_t = \perp$ 。

SlateQ 不直接更新整个推荐列表的价值 $Q(s_t, A_t)$, 而是更新用户实际选择的物品对应的物品级价值 $Q(s_t, c_t)$ 。对于用户选择了物品 c_t 的样本, TD 目标可以写成

$$y_t = r_t + \gamma \max_{A' \subseteq \mathcal{I}, |A'|=k} Q(s_{t+1}, A').$$

根据 SlateQ 的价值分解, 下一状态下的最优列表价值为

$$\max_{A' \subseteq \mathcal{I}, |A'|=k} Q(s_{t+1}, A') = \max_{A' \subseteq \mathcal{I}, |A'|=k} \sum_{i \in A'} P(i | s_{t+1}, A') Q(s_{t+1}, i).$$

若采用 MNL 选择模型, 则可以进一步写为

$$\max_{A' \subseteq \mathcal{I}, |A'|=k} \frac{\sum_{i \in A'} v(s_{t+1}, i) Q(s_{t+1}, i)}{v(s_{t+1}, \perp) + \sum_{j \in A'} v(s_{t+1}, j)}.$$

设物品级价值函数由参数为 θ 的模型 $Q_\theta(s, i)$ 近似, 则训练损失可以写成

$$\mathcal{L}(\theta) = (Q_\theta(s_t, c_t) - y_t)^2.$$

在深度强化学习实现中, 通常还会使用目标网络来稳定训练。若目标网络参数为 θ^- , 则 TD 目标可写为

$$y_t = r_t + \gamma Q_{\theta^-}(s_{t+1}, A_{t+1}^*),$$

其中

$$A_{t+1}^* = \arg \max_{A' \subseteq \mathcal{I}, |A'|=k} Q_{\theta^-}(s_{t+1}, A').$$

8.7 算法流程

SlateQ 的整体流程可以概括如下。

1. 收集用户交互数据, 得到样本

$$(s_t, A_t, c_t, r_t, s_{t+1}).$$

2. 根据历史曝光和点击数据, 学习或估计用户选择模型

$$P(i | s, A).$$

在 MNL 模型下, 需要估计每个物品的吸引力函数 $v(s, i)$ 。

3. 使用神经网络或其他函数近似器学习物品级长期价值函数

$$Q_\theta(s, i).$$

4. 对于下一状态 s_{t+1} , 根据当前的物品级价值函数和用户选择模型, 求解最优推荐列表

$$A_{t+1}^* = \arg \max_{A' \subseteq \mathcal{I}, |A'|=k} \sum_{i \in A'} P(i | s_{t+1}, A') Q_{\theta^-}(s_{t+1}, i).$$

5. 构造 TD 目标

$$y_t = r_t + \gamma \sum_{i \in A_{t+1}^*} P(i | s_{t+1}, A_{t+1}^*) Q_{\theta^-}(s_{t+1}, i).$$

6. 最小化 TD 误差

$$\mathcal{L}(\theta) = (Q_{\theta}(s_t, c_t) - y_t)^2.$$

7. 重复上述过程，直到物品级价值函数和推荐策略收敛。

8.8 SlateQ 与传统推荐排序的区别

传统推荐排序方法通常学习的是短期反馈，例如点击率、转化率或观看时长。其目标常写为

$$\hat{y}(s, i) = P(\text{click} | s, i),$$

然后按照预测分数对物品排序。该方法更关注当前推荐带来的即时收益。

SlateQ 则关注长期价值，其物品级价值函数为

$$Q(s, i) = R(s, i) + \gamma \sum_{s'} P(s' | s, i) V(s').$$

因此，SlateQ 关心的是用户消费某个物品之后对未来收益的影响。一个物品即使短期点击率较高，如果它会导致用户后续活跃度下降，其长期价值也可能较低；反之，一个物品短期点击率不一定最高，但如果能够提升用户长期兴趣和留存，则可能具有更高的 Q 值。

同时，SlateQ 显式考虑了推荐列表内部的竞争关系。在 MNL 模型下，加入一个物品会改变分母

$$v(s, \perp) + \sum_{j \in A} v(s, j),$$

从而影响列表中其他物品的选择概率。因此，SlateQ 不是简单地选择单物品分数最高的 k 个物品，而是优化整个列表的期望长期价值。

8.9 SlateQ 的优点与局限

SlateQ 的主要优点包括以下几个方面。

首先，SlateQ 通过价值分解避免了直接学习组合动作价值函数。原始的 slate 动作空间规模为

$$\binom{|\mathcal{I}|}{k},$$

而 SlateQ 只需要学习物品级价值函数 $Q(s, i)$ ，显著降低了动作空间复杂度。

其次，SlateQ 将强化学习中的长期收益引入推荐列表优化。相比于只优化点击率或转化率的监督学习排序模型，SlateQ 能够考虑当前推荐对用户未来行为、长期活跃度和累计收益的影响。

再次，SlateQ 显式引入用户选择模型，使得推荐列表中不同物品之间的竞争关系能够被建模。这比简单的逐物品打分排序更符合实际推荐场景。

但是, SlateQ 也存在一些局限。

第一, SlateQ 依赖用户选择模型的准确性。如果 $P(i | s, A)$ 估计不准确, 则列表级价值 $Q(s, A)$ 的计算也会产生偏差。

第二, SlateQ 依赖奖励和状态转移主要由用户最终选择物品决定的假设。在真实推荐系统中, 即使用户没有点击某些曝光物品, 这些物品也可能影响用户体验和后续行为。因此, 当未点击曝光也会影响用户状态时, SlateQ 的分解假设可能不完全成立。

第三, SlateQ 更适合单次选择或单次消费场景。如果用户会在同一个推荐列表中连续点击多个物品, 或者列表的多样性、新颖性、重复性本身会显著影响用户体验, 则需要对 SlateQ 进行扩展。

第四, SlateQ 仍然面临离线强化学习中的常见问题, 例如分布偏移、价值高估、反事实评估困难等。因此, 在实际应用中通常需要结合离线评估、保守价值估计和在线 A/B 实验进行验证。

8.10 总结

综上, SlateQ 是一种面向推荐列表优化的价值分解型强化学习算法。它的核心思想可以概括为

$$\text{SlateQ} = \text{物品级长期价值学习} + \text{用户选择模型} + \text{推荐列表优化}.$$

具体而言, SlateQ 不直接学习整个推荐列表的动作价值 $Q(s, A)$, 而是在用户选择模型 $P(i | s, A)$ 的帮助下, 将列表价值分解为

$$Q(s, A) = \sum_{i \in A} P(i | s, A) Q(s, i).$$

其中, $Q(s, i)$ 表示用户选择并消费物品 i 后产生的长期价值, $P(i | s, A)$ 表示物品 i 在当前推荐列表中被用户选择的概率。在 MNL 选择模型下, 列表优化进一步转化为

$$\max_{A: |A|=k} \frac{\sum_{i \in A} v(s, i) Q(s, i)}{v(s, \perp) + \sum_{j \in A} v(s, j)}.$$

因此, SlateQ 既避免了直接处理巨大组合动作空间的困难, 又能够在推荐排序中引入长期收益优化思想, 是强化学习应用于推荐系统中 slate recommendation 问题的一种典型范式。